

Exercise 1: Implementing the Singleton Pattern

```
Logger.java 1 X  LoggerTest.java 1
SingletonPatternExample > Logger.java > Logger
1 public class Logger {
2     private static Logger instance;
3
4     private Logger(){}
5
6     public static Logger getInstance(){
7         if(instance==null){
8             instance=new Logger();
9         }
10        return instance;
11    }
12
13    public void log(String message){
14        System.out.println("LOG: "+message);
15    }
16 }
17 }
```

```
SingletonPatternExample > LoggerTest.java > LoggerTest
1 public class LoggerTest {
2
3     public static void main(String[] args) {
4         Logger logger1=Logger.getInstance();
5         Logger logger2=Logger.getInstance();
6
7         logger1.log(message: "Application started");
8         logger2.log(message: "User logged in");
9
10        if(logger1==logger2){
11            System.out.println("Both references point to the same Logger instance.");
12        }else{
13            System.out.println("Different Logger instances exist.");
14        }
15    }
16 }
17 }
```

Run | Debug

Logger SingletonPatternExample.Logger.getInstance()
Source: WEEK_1_MANDATORY_5cb347a4

RESULT :

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - SingletonPatternExample + - [ ] [ ] ... |
• PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\SingletonPatternExample> javac *.java
• PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\SingletonPatternExample> java LoggerTest
LOG: Application started
LOG: User logged in
Both references point to the same Logger instance.
❖ PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\SingletonPatternExample> [ ]
```

Exercise 2: Implementing the Factory Method Pattern

```
FactoryMethodPatternExample > J Document.java > Document
1 public interface Document {
2     String getType();
3 }
```

```
J Document.java 1 J WordDocument.java 1 X J PdfDocument.java 1
FactoryMethodPatternExample > J WordDocument.java > WordDocument
1 public class WordDocument implements Document {
2
3     public String getType(){
4         return "Word Document";
5     }
6 }
```

```
J Document.java 1 J WordDocument.java 1 J PdfDocument.java 1 X J ExcelDocument.java 1
FactoryMethodPatternExample > J PdfDocument.java > PdfDocument
1 public class PdfDocument implements Document {
2
3     public String getType(){
4         return "PDF Document";
5     }
6 }
```

```
java 1 J WordDocument.java 1 J PdfDocument.java 1 J ExcelDocument.java 1 X
FactoryMethodPatternExample > J ExcelDocument.java > ExcelDocument
1 public class ExcelDocument implements Document {
2
3     public String getType(){
4         return "Excel Document";
5     }
6 }
```

```

FactoryMethodPatternExample > J ExcelFactory.java > ExcelFactory
1 public class ExcelFactory extends DocumentFactory {
2
3     public Document createDocument(){
4         return new ExcelDocument();
5     }
6 }

```

```

FactoryMethodPatternExample > J DocumentFactory.java > DocumentFactory
1 public abstract class DocumentFactory {
2     public abstract Document createDocument();
3 }

```

```

Document.java 1 | DocumentFactory.java 1 | WordFactory.java 1 X | PdfFactory
FactoryMethodPatternExample > J WordFactory.java > WordFactory
1 public class WordFactory extends DocumentFactory {
2
3     public Document createDocument(){
4         return new WordDocument();
5     }
6 }

```

```

Document.java 1 | DocumentFactory.java 1 | WordFactory.java 1 | PdfFactory
FactoryMethodPatternExample > J PdfFactory.java > PdfFactory
1 public class PdfFactory extends DocumentFactory {
2
3     public Document createDocument(){
4         return new PdfDocument();
5     }
6 }

```

```

1 | WordFactory.java 1 | PdfFactory.java 1 | ExcelFactory.java 1 | FactoryDemo.java 1 X
FactoryMethodPatternExample > J FactoryDemo.java > FactoryDemo
1 public class FactoryDemo {
2
3     Run | Debug
4     public static void main(String[] args){
5
6         DocumentFactory wordFactory=new WordFactory();
7         DocumentFactory pdfFactory=new PdfFactory();
8         DocumentFactory excelFactory=new ExcelFactory();
9
10        Document d1=wordFactory.createDocument();
11        Document d2=pdfFactory.createDocument();
12        Document d3=excelFactory.createDocument();
13
14        System.out.println("Created: "+d1.getType());
15        System.out.println("Created: "+d2.getType());
16        System.out.println("Created: "+d3.getType());
17    }
18 }

```

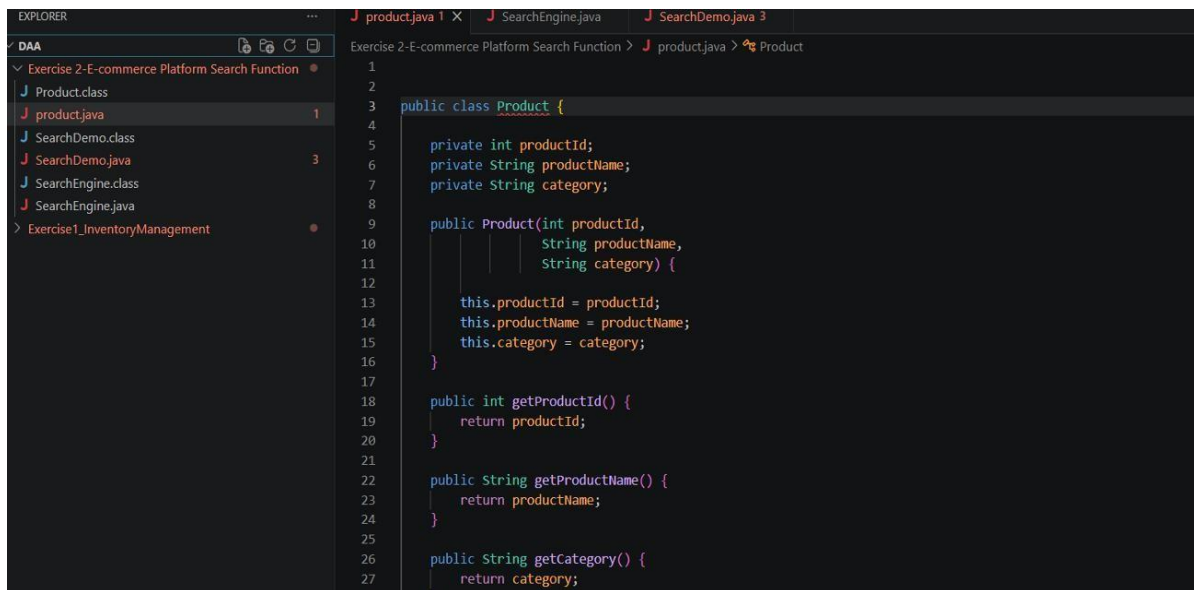
RESULT :

```

PROBLEMS 11 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS
powershell - FactoryMethodPatternExample + - [ ] [ ] [ ] [ ] [ ]
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\FactoryMethodPatternExample> javac *.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\FactoryMethodPatternExample> java FactoryDem
Created: Word Document
Created: PDF Document
Created: Excel Document
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\WEEK_1_MANDATORY\FactoryMethodPatternExample>

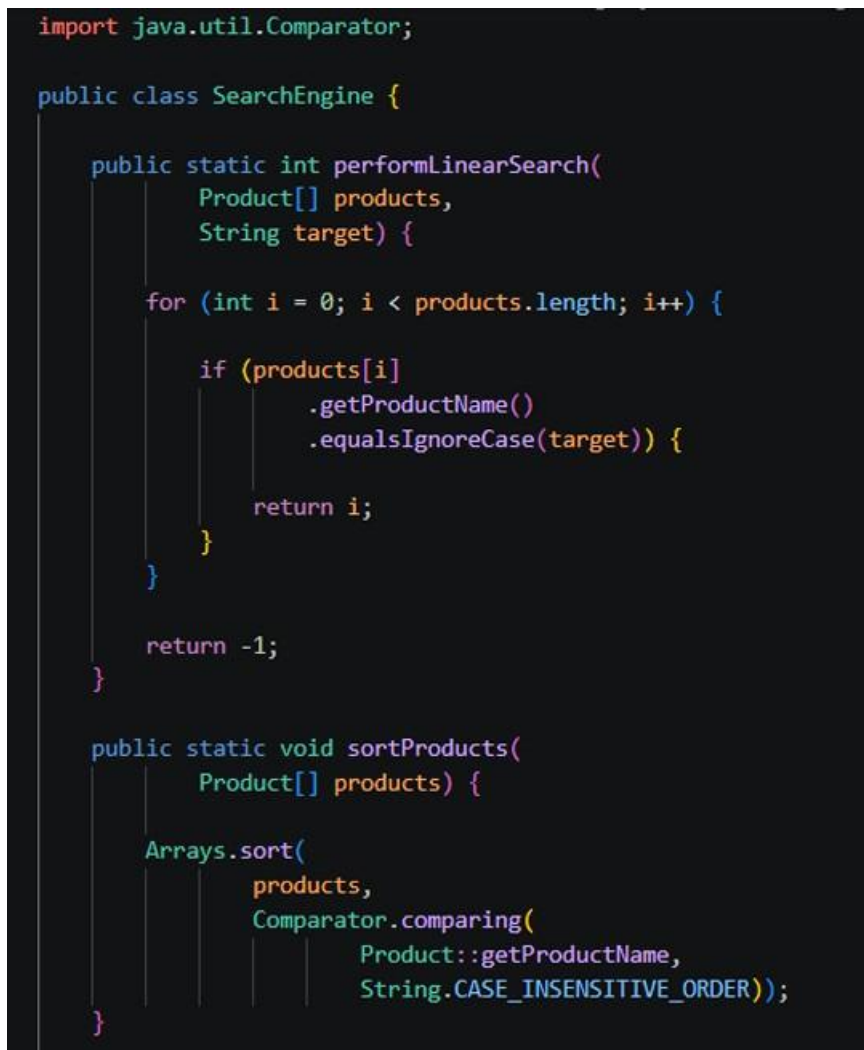
```

Exercise 2: E-commerce Platform Search Function



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'DAA' with a sub-project 'Exercise 2-E-commerce Platform Search Function'. Inside this sub-project, there are files: 'Product.class', 'product.java' (selected), 'SearchDemo.class', 'SearchDemo.java', 'SearchEngine.class', and 'SearchEngine.java'. The code editor displays the implementation of the 'Product' class in 'product.java'.

```
1
2
3 public class Product {
4
5     private int productId;
6     private String productName;
7     private String category;
8
9     public Product(int productId,
10                    String productName,
11                    String category) {
12
13         this.productId = productId;
14         this.productName = productName;
15         this.category = category;
16     }
17
18     public int getProductId() {
19         return productId;
20     }
21
22     public String getProductName() {
23         return productName;
24     }
25
26     public String getCategory() {
27         return category;
28     }
29 }
```



The screenshot shows the implementation of the 'SearchEngine' class. It includes an import statement for 'java.util.Comparator' and two static methods: 'performLinearSearch' and 'sortProducts'.

```
import java.util.Comparator;

public class SearchEngine {

    public static int performLinearSearch(
        Product[] products,
        String target) {

        for (int i = 0; i < products.length; i++) {

            if (products[i]
                .getProductName()
                .equalsIgnoreCase(target)) {

                return i;
            }
        }

        return -1;
    }

    public static void sortProducts(
        Product[] products) {

        Arrays.sort(
            products,
            Comparator.comparing(
                Product::getProductName,
                String.CASE_INSENSITIVE_ORDER));
    }
}
```

```

public static int performBinarySearch(
    Product[] products,
    String target) {

    int left = 0;
    int right = products.length - 1;

    while (left <= right) {

        int mid =
            left + (right - left) / 2;

        int result =
            products[mid]
                .getProductName()
                .compareToIgnoreCase(target);

        if (result == 0) {
            return mid;
        }

        if (result < 0) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}

```

The screenshot shows an IDE with a project named "Exercise 2-E-commerce Platform Search Function". The project contains several files: Product.class, product.java, SearchDemo.class, SearchDemo.java, SearchEngine.class, and SearchEngine.java. The "SearchDemo.java" file is selected, and its content is displayed in the editor. The code defines a public class SearchDemo with a main method that demonstrates the search functionality. It creates an array of Product objects, sets a target string "mouse", and prints the results of both linear and binary searches. The binary search is performed on a sorted array of products.

```

Exercise 2-E-commerce Platform Search Function > SearchDemo.java > SearchDemo > main(String[])
3 public class SearchDemo {
4
5     Run | Debug
6     public static void main(String[] args) {
7         Product[] products = new Product[] {
8             new Product(productId: 1, productName: "Laptop", category: "Electronics"),
9             new Product(productId: 2, productName: "Mouse", category: "Electronics"),
10            new Product(productId: 3, productName: "Keyboard", category: "Electronics"),
11            new Product(productId: 4, productName: "Smartphone", category: "Electronics"),
12            new Product(productId: 5, productName: "Headphones", category: "Electronics")
13        };
14
15        String target = "mouse";
16
17        System.out.println(x: "Linear Search (unsorted):");
18        int idx1 = SearchEngine.performLinearSearch(products, target);
19        System.out.println("Target: " + target + " -> index = " + idx1);
20
21        // Binary search requires sorted array
22        System.out.println(x: "\nSorting products by name...");
23        SearchEngine.sortProducts(products);
24
25        System.out.println(x: "Binary Search (sorted):");
26        int idx2 = SearchEngine.performBinarySearch(products, target);
27        System.out.println("Target: " + target + " -> index = " + idx2);
28
29        if (idx2 != -1) {
30            System.out.println("Found product: " + products[idx2].getProductName()
31                + " (category=" + products[idx2].getCategory() + ")");
32        }
33    }
34 }

```

RESULT :

```
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> cd C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function; javac Product.java SearchEngine.java SearchDemo.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function>
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> java SearchDemo
Linear Search (unsorted):
Target: mouse -> index = 1

Sorting products by name...
Binary Search (sorted):
Target: mouse -> index = 3
Found product: Mouse (category=Electronics)
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise 2-E-commerce Platform Search Function> |
```

Exercise 7: Financial Forecasting

```
J ForecastCalculator.java 1 J ForecastDemo.java 1 X
Exercise7_FinancialForecasting > J ForecastDemo.java > ForecastDemo
1 public class ForecastDemo {
2
3     Run | Debug
4     public static void main(String[] args){
5
6         double currentAmount=50000;
7         double growthRate=0.08;
8         int years=5;
9
10        double futureValue=
11            ForecastCalculator.predictValue(
12                currentAmount,
13                growthRate,
14                years
15            );
16
17        System.out.println("Current Amount : "+currentAmount);
18        System.out.println("Growth Rate : "+(growthRate*100)+"%");
19        System.out.println("Years : "+years);
20        System.out.println("Predicted Future Value : "+futureValue);
21    }
22 }
```

```
J ForecastCalculator.java 1 X J ForecastDemo.java 1
Exercise7_FinancialForecasting > J ForecastCalculator.java > ForecastCalculator
1 public class ForecastCalculator {
2
3     public static double predictValue(double amount,double growthRate,int years){
4         if(years==0)
5             return amount;
6
7         return predictValue(amount*(1+growthRate),growthRate,years-1);
8     }
9 }
```

RESULT:

```
PROBLEMS 28 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> javac *.java
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> java ForecastDemo
Current Amount : 50000.0
Growth Rate : 8.0%
Years : 5
Predicted Future Value : 73466.40384000003
PS C:\Users\Kanna\OneDrive\Desktop\COGNIZANT\DEEPAK\WEEK1\DAA\Exercise7_FinancialForecasting> 
```